Python Programming Fundamentals

is-a vs has-a Relationships

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Two fundamental object relationships

Relationship	Meaning	Implementation
is-a	Child **is a type of** parent	Inheritance
has-a	Class **contains** another class	Composition

Examples:

- A `PremiumMember` **is-a** `Person` ⟶ Inheritance
- A `Shop` **has-a** list of `Products` ⟶ Composition
- A `Car` **has-a** `Engine` ⟶ Composition (not inheritance!)

Outline

2. isinstance() and issubclass()

```
alice = PremiumMember("Alice", "alice@example.com", "Gold")

isinstance(alice, PremiumMember)  # True
isinstance(alice, Person)         # True  ← also a Person!
isinstance(alice, FreeUser)       # False

issubclass(PremiumMember, Person)    # True
issubclass(FreeUser, Person)         # True
issubclass(Person, PremiumMember)    # False (wrong direction)
```

isinstance is very useful for type checking in functions:

```
def process(user):
    if isinstance(user, PremiumMember):
        grant_premium_access()
```

Outline

3. Composition Example

```
class Engine:
    def __init__(self, horsepower: int):
        self.__hp = horsepower
    def start(self): return "Vroom!"

class Car:
    def __init__(self, make: str, hp: int):
        self.__make = make
        self.__engine = Engine(hp)   # has-a Engine

    def drive(self):
        return self.__engine.start()
```

`Car` does NOT inherit from `Engine`. It **contains** an `Engine`. This is composition.

Outline

4. When to Use Inheritance

Use inheritance when:

- The is-a relationship is genuine and stable
- The child class really is a more specific version of the parent
- You want the child to be usable anywhere the parent is used

Avoid inheritance when:

- The relationship is really has-a
- You're inheriting just to reuse methods (use composition instead)
- The hierarchy would become deep and confusing

"Favour composition over inheritance" — classic OOP advice

Outline

5. Practical Examples

```python
# Good use of inheritance
class Animal: pass
class Dog(Animal): pass      # Dog is-a Animal ✓
class Cat(Animal): pass      # Cat is-a Animal ✓

# Bad use of inheritance
class Car: pass
class Engine(Car): pass      # Engine is NOT a Car ✗
# Better:
class Car:
    def __init__(self):
        self.__engine = Engine()  # has-a Engine ✓
```

Outline

6. Using isinstance for Safe Processing

```python
def print_user_info(user: Person):
    print(f"Name: {user.get_name()}")
    print(f"Email: {user.get_email()}")

    if isinstance(user, PremiumMember):
        print(f"Plan: {user.get_subscription()}")
    elif isinstance(user, FreeUser):
        print(f"Post limit: {user.get_max_posts()}")

# Works for any subclass of Person
for user in all_users:
    print_user_info(user)
    print("---")
```

Thanks for Watching - Any questions?